

SuperBizAgent Python 项目

整体架构、信息流与模块职责说明

基于 P0-P3 改造完成后的当前代码

文档目的	帮助新成员由浅入深理解系统边界、主流程、状态和依赖
分析范围	应用入口、前后端、工作流、数据层、外部系统、测试与运行方式
生成日期	2026-08-05
项目目录	C:\zyh\OnCallAgent\Python-super_biz_agent_py-release-2026-05-17\super_biz_agent_py-release-2026-05-17

一、项目定位

这是一个面向值班运维人员的 AI 运维助手，主要解决告警出现以后，如何查资料、收集证据、分析根因并形成处置建议的问题。

系统同时提供三条业务路径：普通 AI 对话和知识问答、可以暂停与恢复的持久化 AIOps 诊断、按固定角色分工的企业多智能体事故分析。

后端主要使用 FastAPI、LangGraph、LangChain、Qwen/DashScope、PostgreSQL 和 Milvus；前端是原生 HTML、CSS、JavaScript，没有 Vue、React 等前端框架。

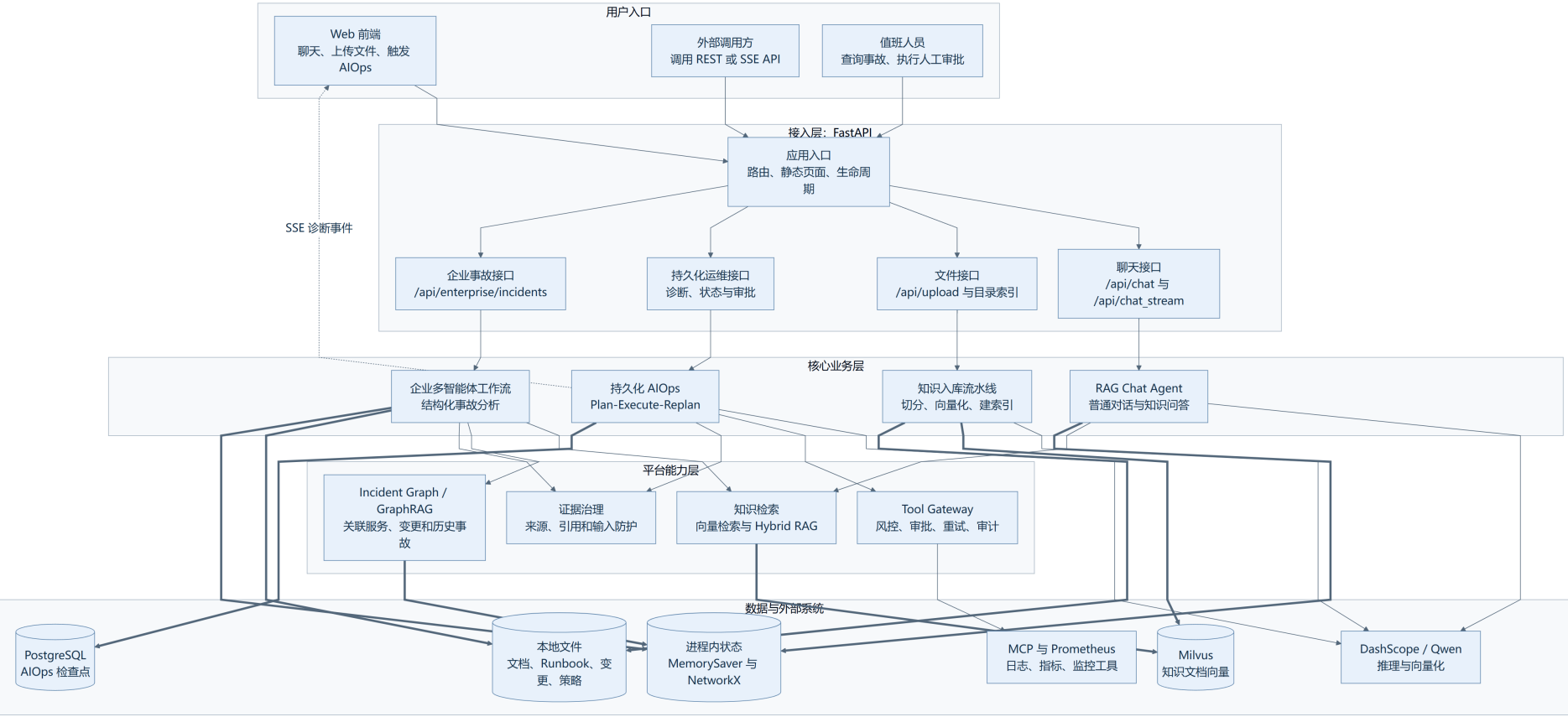
应用由 Uvicorn 启动。FastAPI 在启动阶段连接 Milvus 和 PostgreSQL，并通过 REST API 或 SSE 流向浏览器返回结果。

阅读顺序

- 先看系统总览，记住 FastAPI 是总入口，后面有三条主要业务路径。
- 再看聊天与知识入库，理解普通问答和 Milvus 知识库的关系。
- 然后看持久化 AIOps，理解 Planner、Executor、审批与 PostgreSQL 恢复。
- 接着看企业工作流，理解多个角色如何按固定顺序协作。
- 最后看启动与存储边界，确认哪些数据会保留，哪些只存在内存。

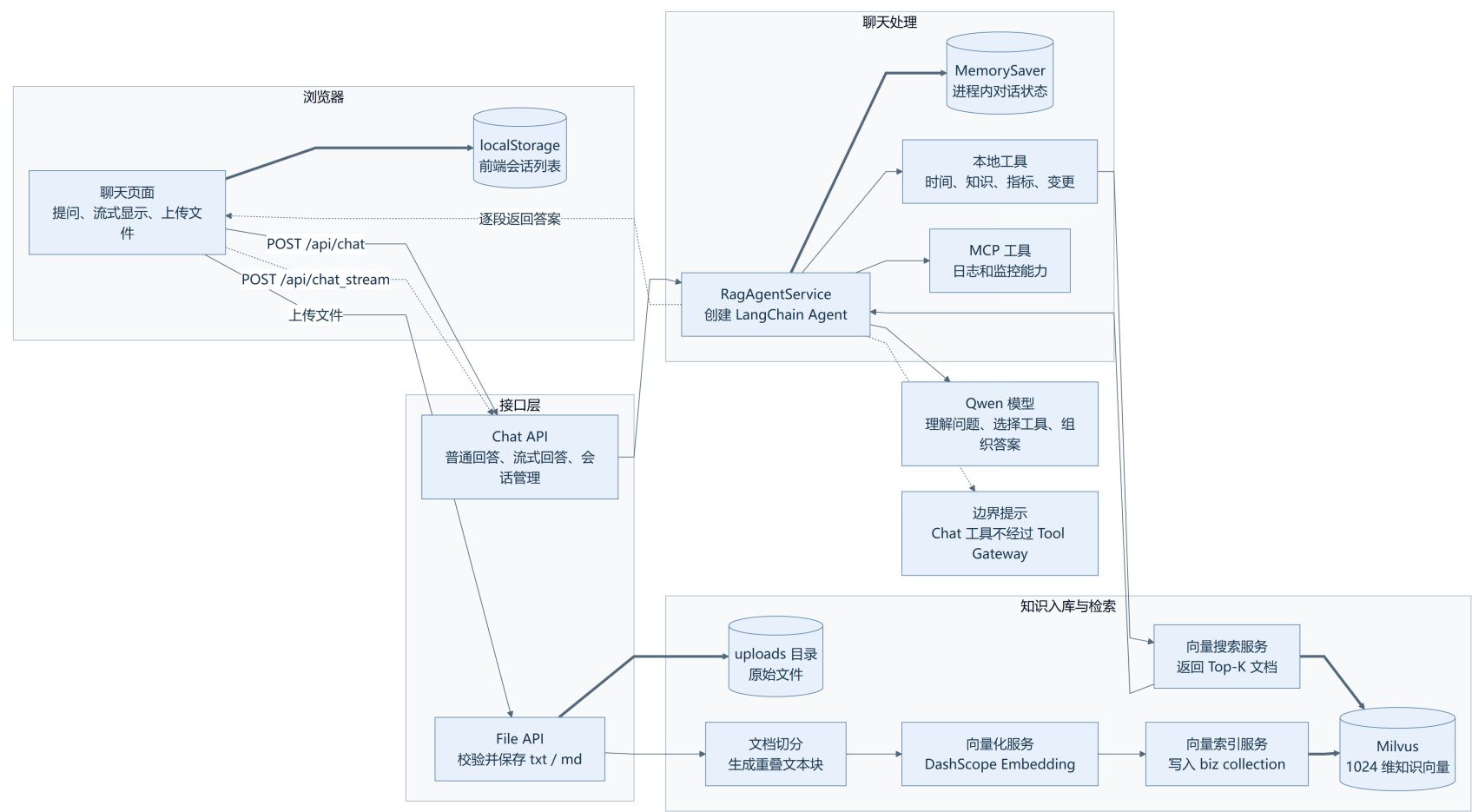
图例：实线箭头表示同步 HTTP 或程序内部调用；虚线箭头表示 SSE、并行流程或人工中断与恢复；粗线箭头表示数据库、文件或状态读写。

二、整体结构图 - 系统总览



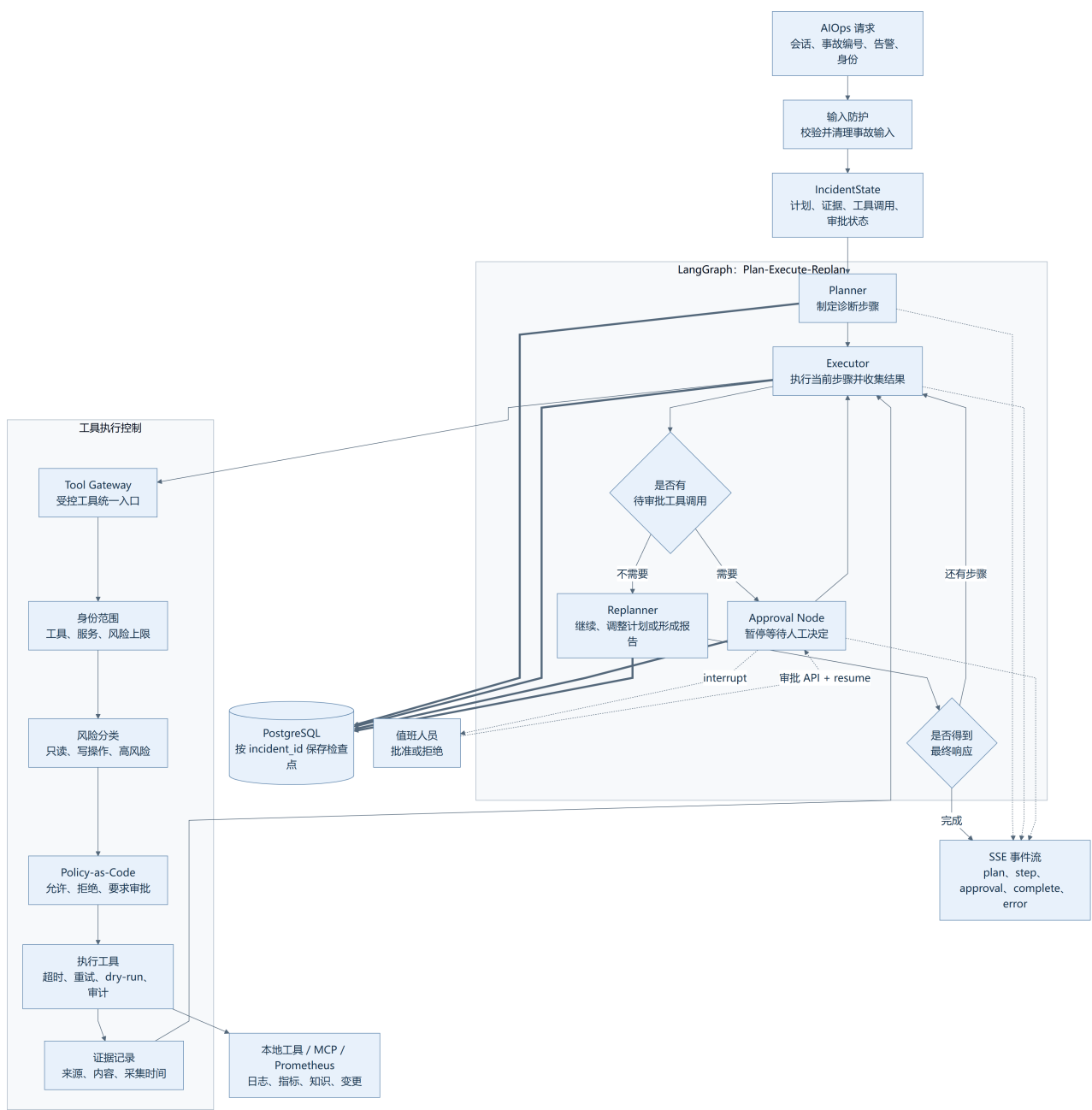
大白话理解：FastAPI 是总接待台。普通聊天、持久化 AIOps 和企业事故工作流是三个独立办事部门，它们共享模型、检索和外部工具，但保存状态的方式不同。

子图一：普通聊天与知识入库



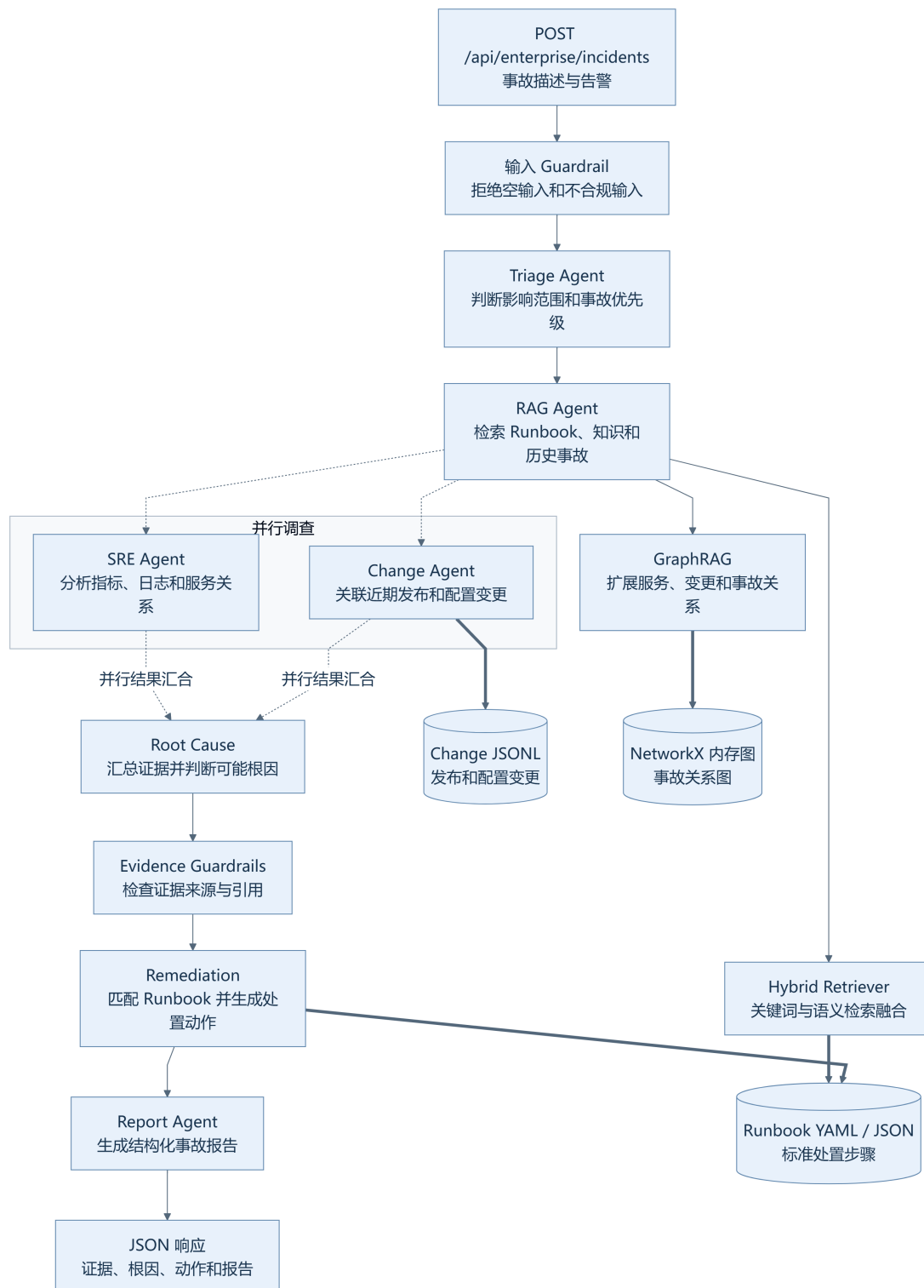
普通聊天使用进程内 MemorySaver；浏览器另存一份 localStorage 历史。上传文档经过切分与向量化后进入 Milvus。Chat Agent 直接绑定工具，不经过 Tool Gateway。

子图二：持久化 AIOps 诊断



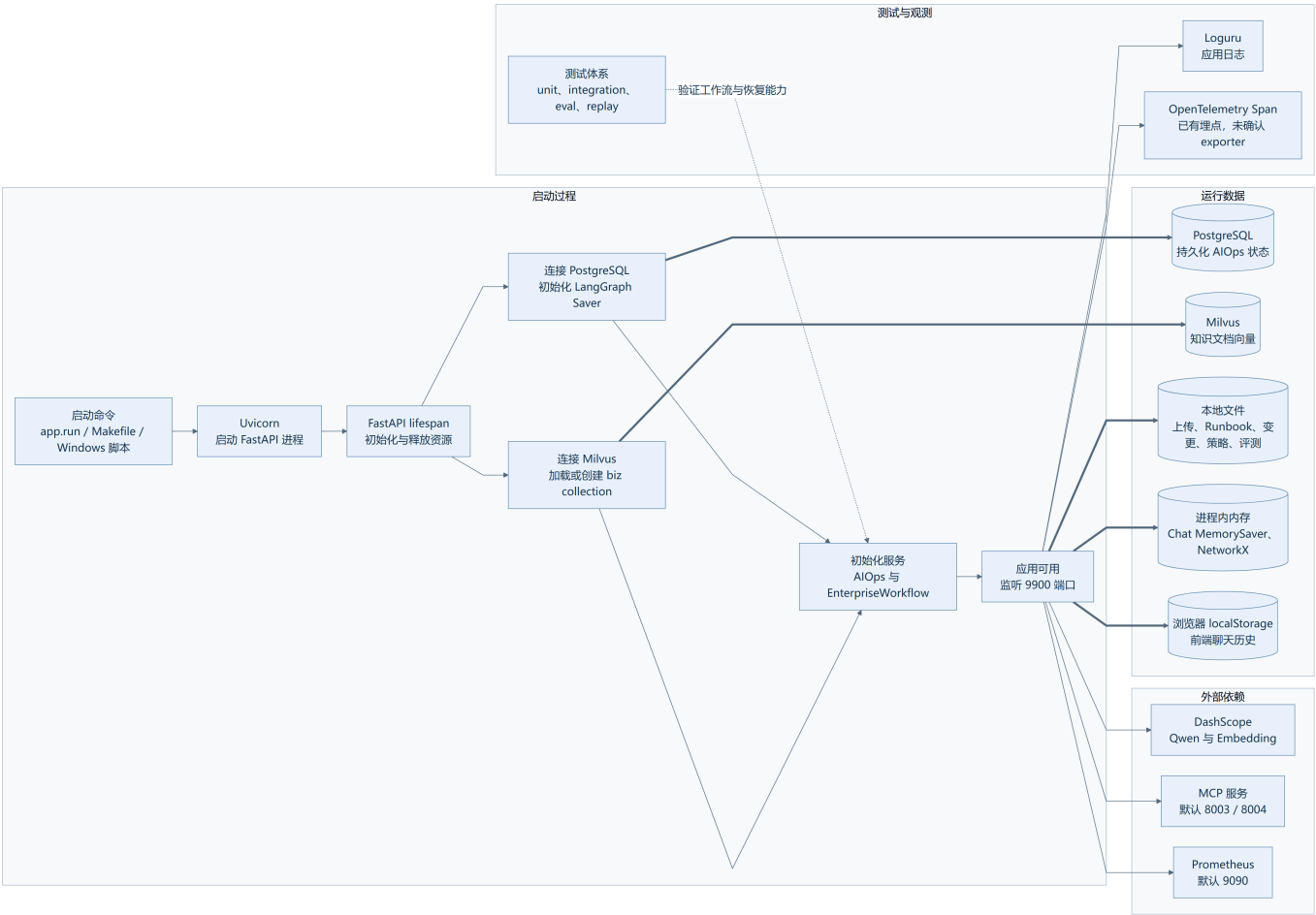
Planner 制定计划，Executor 逐步执行，Replanner 决定继续或生成报告。需要人工审批时，LangGraph 将当前状态写入 PostgreSQL 并暂停，审批后从同一事故检查点恢复。

子图三：企业多智能体事故分析



流程按固定角色推进：Triage -> RAG -> SRE/Change 并行 -> Root Cause -> Remediation -> Report。当前图数据和部分角色证据以进程内或演示数据为主。

子图四：启动、存储与运行边界



应用启动依赖 Milvus 和 PostgreSQL。PostgreSQL 保存 AIOps 状态，Milvus 保存知识向量；Chat MemorySaver 和 NetworkX 只存在当前进程。当前没有 Redis、Kafka 或 RabbitMQ。

三、核心调用链

1. 普通聊天

浏览器 -> Chat API -> RagAgentService -> Qwen -> 本地或 MCP 工具 -> 返回答案

主要模块：static/app.js、app/api/chat.py、app/services/rag_agent_service.py、app/tools/、app/agent/mcp_client.py。

2. 文档进入知识库

上传文件 -> File API -> uploads 目录 -> 文档切分 -> DashScope Embedding -> Milvus

主要模块：app/api/file.py、document_splitter_service.py、vector_embedding_service.py、vector_index_service.py、vector_store_manager.py。

3. 持久化 AIOps

AIOps API -> Planner -> Executor -> Tool Gateway -> 证据 -> Replanner -> PostgreSQL 检查点 -> SSE 返回

需要审批时：Executor -> Approval Node -> PostgreSQL 暂存 -> 人工审批 -> Command resume -> Executor。incident_id 同时作为 LangGraph thread_id；trace_id 标识一次执行链。

4. 企业多智能体分析

Enterprise API -> Triage -> RAG -> SRE 与 Change 并行 -> Root Cause -> Remediation -> Report

主要模块：app/agent/enterprise_workflow.py、app/retrieval/hybrid.py、app/incident_graph/、app/runbooks.py、app/change_intelligence.py。

5. 应用启动

app.run -> Uvicorn -> FastAPI lifespan -> 连接 Milvus -> 连接 PostgreSQL -> 初始化 workflow -> 开放接口

Milvus 或 PostgreSQL 连接失败都会影响当前应用初始化。

四、三个业务路径的关键差异

路径	主要用途	状态保存	工具治理
普通聊天	问答、知识检索	MemorySaver + 浏览器 localStorage	工具直接绑定，不经过 Tool Gateway
持久化 AIOps	生产式告警诊断与审批	PostgreSQL LangGraph 检查点	身份、风险、策略、dry-run、审批、审计
企业工作流	结构化多角色事故分析	单次运行状态 + NetworkX 内存图	证据 Guardrail；不是持久化审批流

- 要长期保存和恢复事故进度，应走持久化 AIOps，而不是普通聊天。
- 需要固定角色、固定报告结构和可重复评测时，企业工作流更合适。
- 普通聊天适合快速问答，但当前工具治理强度低于持久化 AIOps。

五、模块职责表

模块/子系统	核心职责	关键目录或文件	主要依赖
应用入口	注册路由、静态资源、生命周期	app/main.py; app/run.py	FastAPI; Uvicorn
Web 前端	聊天、上传、AIOps 触发和本地历史	static/index.html; static/app.js	Fetch; SSE; localStorage
Chat API	普通和流式对话、会话查询与清理	app/api/chat.py	RagAgentService
文件 API	上传文档、触发单文件或目录索引	app/api/file.py	VectorIndexService
AIOps API	诊断、状态查询、审批、企业事故入口	app/api/aiops.py	AIOpsService; EnterpriseWorkflow
Chat Agent	调用模型和工具完成普通问答	app/services/rag_agent_service.py	Qwen; MemorySaver; MCP
持久化 AIOps	编排计划、执行、审批和重新规划	app/services/aiops_service.py	LangGraph; PostgreSQL
AIOps 节点	实现 Planner、Executor、Replanner	app/agent/aiops/	Qwen; Tool Gateway
Tool Gateway	工具注册、风控、审批、超时和审计	app/agent/tool_gateway.py	Policy; Identity; Risk
证据治理	统一证据结构、引用和输入检查	app/agent/evidence.py	IncidentState
企业 workflow	结构化多角色事故分析	app/agent/enterprise_workflow.py	Hybrid RAG; GraphRAG
知识检索	向量、关键词和融合检索	vector_search_service.py; app/retrieval/hybrid.py	Milvus; 文档集合
Incident Graph	服务、事故、变更关系存储和查询	app/incident_graph/	NetworkX
Runbook	加载和匹配标准处置流程	app/runbooks.py	YAML; JSON
变更智能	关联事故与近期发布变更	app/change_intelligence.py	JSONL
PostgreSQL 检查点	保存可暂停、可恢复的事故状态	app/core/checkpoint.py	psycopg; LangGraph Saver
Milvus 管理	管理连接、Collection 和向量索引	app/core/milvus_client.py	pymilvus
评测与回放	验证检索质量和历史故障行为	app/evals/; app/replay/; tests/	pytest; JSON 数据

六、架构说明

分层说明：本项目不是严格的 Controller -> Service -> Repository 架构。app/api 相当于 Controller；app/services 与 app/agent 共同承担业务层；checkpoint.py、milvus_client.py 和 vector_store_manager.py 相当于数据访问层；当前没有统一 Repository 接口。

模块划分原则

系统主要按使用场景划分。普通聊天、持久化 AIOps 和企业事故分析拥有独立的状态与编排方式，而不是强制共享一套 workflow。

核心依赖方向

正常方向为：浏览器或 API -> 路由 -> Service/Workflow -> 工具与检索 -> 数据库或外部系统。API 层主要负责参数接收、依赖获取、错误转换和流式返回。

关键数据流

聊天消息保存在浏览器 localStorage 与后端 MemorySaver；知识文档进入 Milvus；AIOps IncidentState 按 incident_id 写入 PostgreSQL；企业事故状态主要在一次 workflow 运行中传递。

系统边界

项目内部负责 AI 编排、状态、检索和工具治理。Qwen、MCP、Prometheus、PostgreSQL、Milvus 属于外部运行依赖。系统没有 Redis、Kafka、RabbitMQ 等缓存或消息队列。

高耦合或重点关注项

- RagAgentService 直接绑定工具，绕过 Tool Gateway 的身份、风险、审批和审计控制。
- 普通聊天同时依赖浏览器历史与后端 MemorySaver，两份状态可能出现不一致。
- VectorStoreManager 的全局实例可能在模块导入阶段连接 Milvus，增加启动和测试耦合。
- 企业 workflow 默认 GraphRAG 数据、角色证据和 NetworkX 图带有演示性质。
- 前端没有人工审批界面，也没有企业事故 workflow 入口。
- AIOps 的持久化依赖 PostgreSQL，但现有 Windows 启动脚本和 Makefile 没有统一启动 PostgreSQL Compose。

七、依据与不确定项

关键代码依据

- app/main.py：应用生命周期、路由与服务初始化。
- app/api/aiops.py：AIOps、事故查询、人工审批和企业接口。
- app/services/aiops_service.py：持久化 workflow 拓扑与恢复方式。

- app/agent/tool_gateway.py: 工具治理边界。
- app/agent/enterprise_workflow.py: 企业多智能体流程。
- app/services/rag_agent_service.py: 普通聊天、MemorySaver 和工具绑定。
- app/core/checkpoint.py: PostgreSQL LangGraph 检查点。
- static/app.js: 前端请求、SSE 和 localStorage。
- docs/learning/README.md: P0-P3 能力和验收索引。

待确认事项

- 待确认: 生产部署拓扑。仓库中的本地启动和 Compose 文件不足以证明使用 Kubernetes、虚拟机或特定云平台。
- 待确认: 生产 MCP 服务的真实能力。代码能确认协议和地址, 不能确认外部服务返回的日志范围、监控范围与权限。
- 待确认: OpenTelemetry 数据发送位置。已有 Span 埋点, 但没有发现 exporter、collector 或观测平台配置。
- 待确认: PostgreSQL 的生产备份与高可用策略。当前代码只负责连接池和检查点读写。
- 待确认: 人工审批的真实操作入口。后端 API 已存在, 但网页端没有审批页面。

验证记录: 非 PostgreSQL 测试套件为 115 passed, 代码覆盖率 64.18%。PostgreSQL 持久化集成测试仍需要实际数据库环境完成独立验证。